Institute of Engineering and Technology (IET)

Minor Project Synopsis(PR1103)

Design and Development of an Intelligent Conversational Voice Agent for Customer Care Automation

PREPARED BY

Kanishk Jain (2023Btech040)
Tanveer Kanderiya (2023Btech091)

FACULTY GUIDE

Mr. Santosh Verma

Jan, 2026

Table Of Content **2**

1. Abstract 3
2. Introduction 4
3. Problem Statement 4
4. Objectives 4
5. Scope of Work 5
6. Methodology 5-8
   - Backend System Design and Setup
   - Conversion of Spoken Language into Written Text
   - Detection and Classification of Intentions
   - Finite State Machines Control Dialogue
   - Connecting the Business Logic with the Data Layer
   - Text-to-Speech
   - Telephony
   - Controlled Use of Large Language Models
   - Testing and Validation
   - Development Timeline Alignment
7. Technology Stack 8
8. Timeline 9
9. Differentiation from Existing Solutions (Unique Selling Propositions) 9-10
10. Expected Outcomes 10
11. Future Enhancements 10
12. Conclusion 10
13. References 10

1. Abstract

The area of customer care is one of the most important operational areas of an organization, especially in a market like India where conversations are primarily in the form of spoken words. Although there are many modern digital communication channels available to customers, the majority of customer queries are still made over the telephone via a human operator. For most small to medium enterprises in India, they depend primarily on old IVR systems & phone operators which come with their own disadvantages, the length of the wait time, how rigidly you must navigate through the menu, the length of time you have to talk about the same topic over and over again, the costs of operating the business when you are relying on a person to handle each call.

To develop a better experience for customers that calls into a business to get help or answers to their questions, a smart virtual agent will be developed to answer some of the same questions that human operators would normally handle through voice interactive technology using natural language understanding. The new system will take spoken input from the caller, understand the intent behind the caller's question, carry the context of conversation for multiple turns, and answer repetitive or routine customer care questions with little or no need for human intervention. While not fully automating all aspects of customer care calls, this will allow for only selective automation of those calls that occur frequently.

This conceptual design implements a modular, end-to-end solution with telephony integration, speech-to-text conversion for natural language understanding, intent detection (what do you want us to do?), State Machine for managing interactions within defined domains via Rules or Policies, Execution of decisions made using those defined Rules or Policies based upon user input, response Generation using the Text-to-Speech framework and delivery mechanism etc. A Hybrid approach is taken where an intelligent machine learning model combined with hard-coded logic that determines the flow of conversation and decision-making based upon user experience. Limited applicability of LLMs will also improve response fluency and resolve ambiguity, this will also remove the risk of hallucinations.

The system follows an India-first design approach, supporting Hindi and English speakers friendly towards accent variation in India by incorporating lower-cost Open Source tools to create this functional prototype; allowing small and medium size businesses to manage a significant percentage of their repetitive customer contact through automated video and phone-based applications.

2. Introduction

Customer service is an incredibly visible point of interaction for customers between companies and the organizations that provide them with goods and services. The way in which customers communicate with companies in India is primarily through the use of voice-based systems.

Many companies that offer customer service in India rely heavily on automated systems for navigating the various menu options. Traditional IVR systems rely on long and rigid menu structures, often resulting in extended waiting times and repeated handoffs between agents.

Additionally, companies face high operating costs associated with maintaining large agent teams, training staff, providing inconsistent and/or low quality of customer service, and not achieving low rates of resolution on the first contact.

Research and industry data show that a large percentage of calls received from customers can be classified as being repetitive and rule-based, for example: order tracking, checking on the status of a refund, initiating a return, activating a service, and scheduling an appointment. Automating these interactions has the potential to lower the company's operational costs and increase the customer experience.

3. Problem Statement

Small and medium-sized Indian companies have many problems with their existing customer support systems:

- Menu-based IVR systems that rely on predefined voice prompts and cannot handle normal human conversations.
- Extended periods of time between calls being initiated by customers and the ability of live agents to answer those calls due to poor call flow design.
- A heavy reliance on human agents to handle repetitive, predictable, and well-defined questions from customers.
- High costs associated with operating support systems and the resulting physical and mental fatigue of customer service representatives.
- An absence of affordable, explainable, and localized conversational voice solutions for Indian companies.

Consequently, there is a requirement for a product that automates repetitive customer servicing interactions by enabling natural voice conversations, while maintaining reliability, security, and cost-effectiveness within the overall customer support process.

4. Objectives

The goals outlined for this initiative are as follows:

- Designing and building a conversational voice interface solution for traditional IVR systems through speech recognition technology.
- Implementing a speech-to-text solution for identifying customer requests based on spoken words.
- Creating mechanisms for managing multi-turn conversations in a structured manner.
- Automating specific high-volume opportunities for customer service personnel.
- Supporting interactions in both English and Hindi.
- Providing a safe way for customers to connect with human agents as necessary.
- Constructing an evaluation system with modular components that allow for an objective assessment of the development work.

5. Scope of Work

Creating a prototype, which can be used by companies in one category (E-commerce). The project's goal is to develop a computerized system that automates common customer service support requests.

Examples of the Customer Service Support Requests include:

- Why has my order not arrived?
- How do I return an item?
- Where is my refund?

The project will not create advanced features, such as those required for use by an enterprise (Company), such as customer relationship management (CRM) integration, billing systems, compliance logs, and deploying large volumes of messages.

6. Methodology

An Incremental Pipeline Approach for Build-out of the Proposed Conversational Voice Agent.

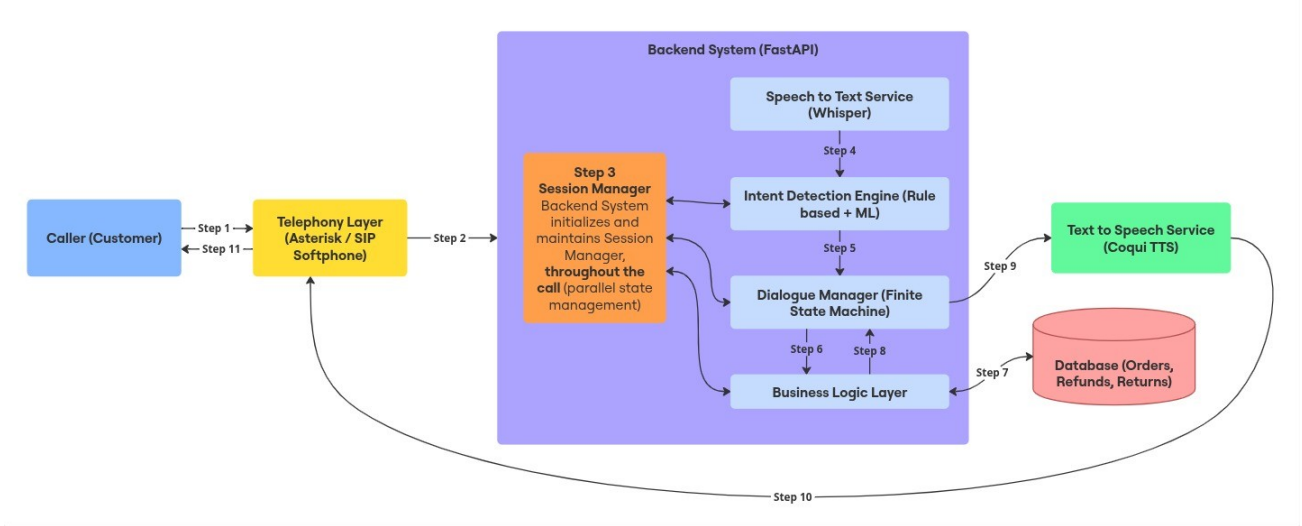
Each system component of the proposed conversational voice agent will be independently designed, implemented, and validated prior to complete integration. The approach taken for developing the system emphasizes Architectural Clarity (understood as clear specifications for each component's expected performance characteristics), Controlled Intelligence (the degree of complexity of the model will be minimised), and Functional Reliability (functional operations must be verified).

The Voice Interaction Pipeline for the complete system is divided into distinct yet interdependent stages. Each stage corresponds with a specific Technical Module.

The sequential stages of Pipeline Development are:

Voice Input → Speech to Text → Intent Detection → Dialogue Management → Business Logic → Text to Speech → Voice Output.

The Incremental Pipeline Approach has been developed through sequential advancement with respect to each stage, thereby allowing for best practices in both Product and Evolution Management for System Integration and Ease of Debugging.



6.1 Backend System Design and Setup

The backend system serves as the primary control centre for the application as a whole, providing access to all incoming calls, state information for active conversations, all of the artificial intelligence functionality, and responses to user input.

FastAPI is chosen to build the backend system because of its lightweight architecture, asynchronous support, and ease of integration with APIs. The first step in building the FastAPI-based backend system is building a minimal backend to test the request flow and session handling capabilities of the system without adding support for artificial intelligence or audio processing.

Some of the primary functions performed by the backend include

- Managing call sessions and call session IDs.
- Routing incoming audio and text inputs to the appropriate module.
- Maintaining the active conversational states between multiple user turns.
- Coordination of the responses generated by the dialogue-based and business logical components.

Initial API endpoints will be built to simulate the life cycle of a call to provide an opportunity to test the business logic prior to developing the voice integration portion of the application.

6.2 Conversion of Spoken Language into Written Text

When the back-end infrastructure is completed, the next stage of processing involves converting the audio signal to written words.

To accomplish this conversion from speech to written words locally on machine resources, the Whisper tool is utilized, as that will relieve any reliance on third-party services, provide predictable response times, and enable the same level of operational expense control as seen with other decomposed services.

The functionality of this speech-to-text service includes the ability to:

- Capture and normalize audio (both recorded and live) formats by sampling them at the same rate
  (Sample Rate) and in the same format (Audio Format)
- Convert the audio into text
- Identify the language used to produce the audio or written output if possible

The speech-to-text tool is an internal web service exposed in our back-end and it is used to create the initial input from a live audio signal during a session. The output of this tool will provide the first of three types of inputs required for all subsequent intent detection and dialog processing.

6.3 Detection and Classification of Intentions

At the heart of the system is a layer of Reasoning that detects the intention of the customer based on the transcription of what the customer said.

The method used to identify the customer’s intention is a hybrid one:

- Utilising rule-based matching of keywords/phrase patterns;
- Using ML to classify with Logistic Regression.

At the beginning of the project, creating a defined set of intentions, e.g. status of order, status of refund, returning an item, and unknown, to avoid any unintended behaviour or changes to the system as a result of customer interaction.

Each customer input is assigned a:

- Predicted Intent Label
- Confidence score (or confidence level).

A low confidence score means that the customer input needs to be fallback or clarification. This helps to avoid service disruption, ensures that automated paths are correctly configured, and increases overall reliability of the system.

6.4 Finite State Machines Control Dialogue

Managing a conversation with a finite state machine keeps the flow steady and predictable, while avoiding sometimes obscure menus in traditional IVR systems. The FSM takes that menu and replaces it with the idea that each step in a dialogue is a state unto itself.

Each action, from greeting to confirming the user's intent and collecting information, until all the required pieces are available, defines a state, a distinct possible action. The dialogue manager takes these states plus user input, database data, and validation rules as input to navigate the conversation from one state to the other. Through it all, the dialogue manager has to keep track of context across turns, remember the order ID and confirmation status (if there is one), decide what to say after an intent confirmation action and what to do if all intent confirmation attempts fail. Required information such as order ID or confirmation details is treated as slot data associated with specific dialogue states.

This design ensures that the system will not output uncontrolled or unsafe results. Using the FSM to construct the dialogue manager, therefore, provides easier reasoning about errors during debugging and the overall evaluation of the system.

6.5 Connecting the Business Logic with the Data Layer

In fact, the business logic level simulates how a customer care operation would function in reality. This runs independently of the natural language processing modules and is triggered only after there is sufficient validated data to move ahead.

We use a lightweight relational database, be it SQLite or PostgreSQL, for storing mock data for orders, refunds, and returns. The database schema is kept simple so that it focuses more on how things work rather than optimizing for database performance.

What is performed in business logic layer includes:

- Fetching order status using order identifiers
- Eligibility for refund and refund status check
- Recording return requests

By decoupling business logic routines from dialogue and intent extraction, it ensures a clean architecture and avoids instances of the AI components affecting the data directly.

6.6 Text-to-Speech

After that, a textual response is designed, then transformed into an auditory format using Coqui TTS. This is TTS, which ensures that interactions between the system and the caller feel more natural because it is voice communication.

What the Text-To-Speech module does:

- Inputs the raw text from the dialogue manager system
- Generates audio files of a pre-defined voice
- Outputs audio ready for telephony playback

This voice features gradual and controlled variations in tone in order to maintain consistency.

6.7 Telephony

We introduce the concept of telephony only after the conversation flow is in place. This ensures that problems in the dialogue logic are not confused with problems in telephony.

To aid development, it allows controlled testing with Asterisk or SIP softphones, enabling us to test call functionalities. Telephony is the entrance and exit route for the system, taking voice input through to the backend system and delivering synthesized voices back to the caller.

6.8 Controlled Use of Large Language Models

Large Language Models are only used in a purely passive manner when there is low confidence in intent detection or the user input is heavily paraphrased.

LLMs cannot:

- Execution of business logic
- Access databases
- Controlling conversation flow

This constrained usage maintains explainability, reduces costs, and makes behavior predictions possible.

6.9 Testing and Validation

We work in small steps progressively in each step:

- Unit testing encompasses intent detection and business logic
- Integration tests examine how dialogue is flowing
- End-to-end tests correspond to real voice calls

We can evaluate the performance of the system based on the accuracy of the answers, the dialog's stability, and the end-to-end latency.

6.10 Development Timeline Alignment

We have a structured step-by-step timeline:

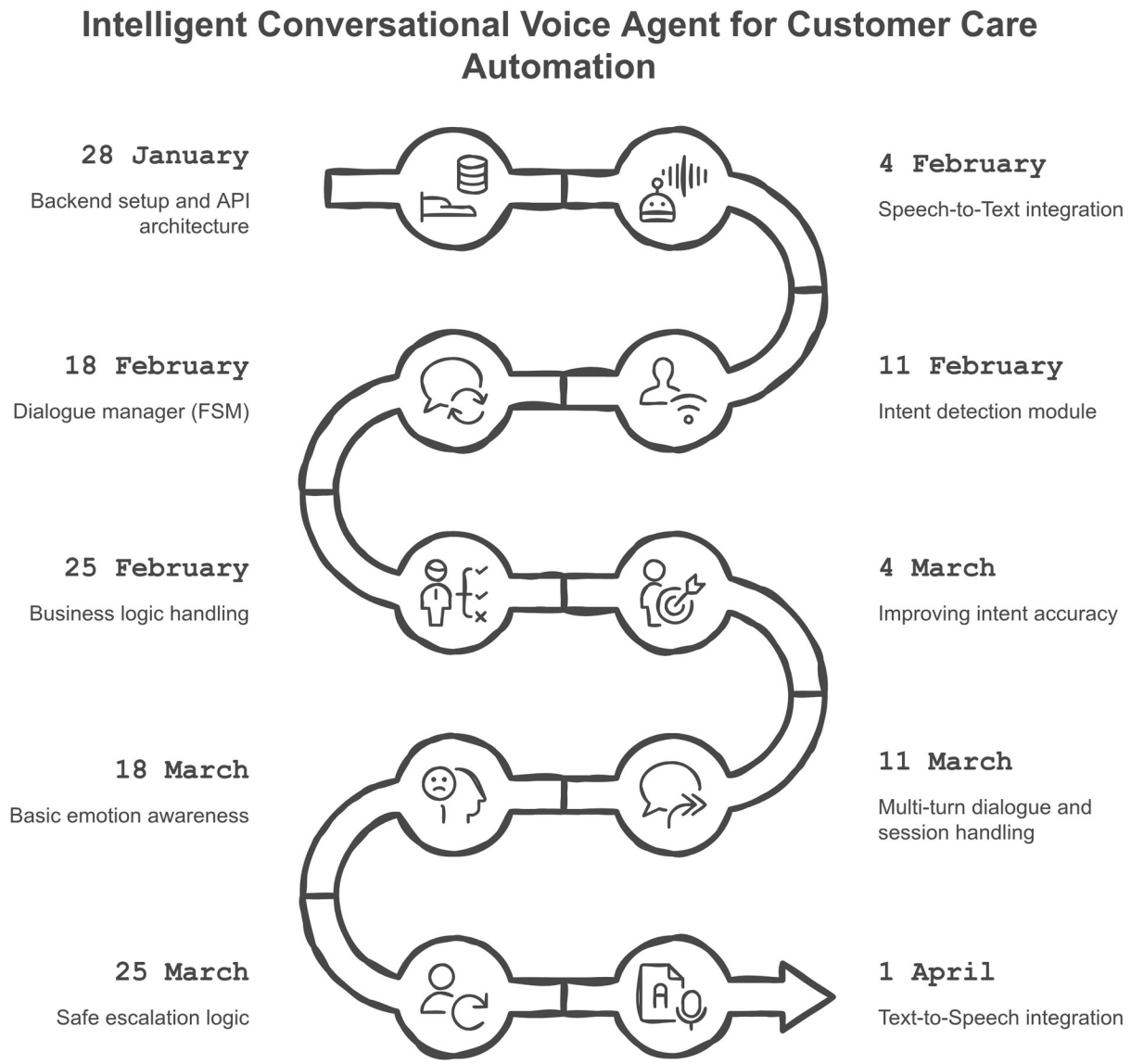
- We handle backend, speech-to-text, intent identification in the early stages.
- Next, the topic moves towards dialogue management and data integration.
- Lastly, telephony integration, testing, and documentation are dealt with.

This helps to keep the system operational and usable throughout the phases of its development.

7. Technology Stack

1. **Telephony:** Asterisk or SIP softphone
   Utilized to test simulations of voice calls in a controlled manner, without needing to use paid telephony.
2. **Backend:** Python with FastAPI
   It acts as the coordinator and deals with call sessions and directs inputs to link all the components.
3. **Speech-to-Text:** Whisper (Local deployment)
   Transcribes audio into text, supports English, Hindi, keeps all operations local, offline, and away from any external API.
4. **Intent Detection:** spaCy and scikit-learn
   Engages in text preprocessing, feature extraction, and intent classification using a hybrid rule-based system combined with machine learning.
5. **Dialogue Management:** Custom Finite State Machine
   Controls the conversation, coordinates multi-turn dialogue, and determines predictable dialogue transition patterns.
6. **Text-to-Speech:** Coqui TTS
   Translates the system responses from the written form to the verbal form that is required in voice interaction with the callers.
7. **Database:** SQLite or PostgreSQL
   This contains mock data regarding order, refund, and return transactions performed on the site.
8. **Frontend Technologies:** Basic HTML/React
   Provides a basic admin interface to monitor calls and analyze the performance of the system.
9. **Deployment Options:** Running on localhost or free cloud tier
   It keeps testing simple and does not involve much infrastructure in the prototype phase. Large
10. **Language Model:** Optional, with limited fall-back usage
    Is used for ambiguous input from the user and fluency. It is not used for any other purpose.

8. Timeline



9. Differentiation from Existing Solutions (Unique Selling Propositions)

This project defies conventions of standard IVR configurations, generic chatbots, and most LLM-based chat agents by adopting a system mentality more focused on outcomes.

What sets it apart:

1. **Conversational over Menu-Based IVR:** The keyboard menu interface might be replaced with a natural language utterance in some customer service situations.
2. **Hybrid intelligence model:** The flow of core decisions and dialogues is based on deterministic rules and finite-state machines. A large language model only supports this process to some extent.
3. **"India-first" design:** Designed keeping in mind the need to support the English and Hindi languages, accent variety, and paving the way for Hinglish and regional languages.
4. **Cost-Aware Architecture:** Open-source solutions and options available for local hosting make it accessible to smaller and midsized businesses with smaller budgets.
5. **Transparent and auditable behavior:** The system has well-structured dialogue management and rule-

based reasoning, making it easy to interpret and repair, which is normally challenging in fully end-to-end LLM solutions.

6. **Full Lifecycle Coverage:** This solution covers the entire path of the customer service call, from telephony integration, speech processing, intent detection, dialogue processing, through to response generation.

10. Expected Outcomes

Towards the conclusion of the project, we are expecting the following outcomes:

1. A functional prototype for an intelligent voice conversational interface suitable for automating customer services.
2. Automation of routine repetitive questions, such as order status, refund status, and starting a return.
3. Reduction in human agents required on well-structured, voluminous interactions.
4. A verifiable way for dialogue management that can be explained through finite state machines.
5. Tips that can be effectively applied in the development of voice AI systems.

11. Future Enhancements

Although the presented system proves that the conversational voice automation is viable in case of certain customer care situations, the following improvements can be discussed in subsequent versions:

- Language and dialect support: Offering support to more Indian languages and enhancing managing of Hinglish and regional dialect variations to make them more accessible.
- Adaptive intent learning: Addition of mechanisms to learn incrementally when an unresolved or escalated call occurs, allowing the intent detection layer to learn better with time, without destabilizing core dialogue logic.
- Awareness of emotion and frustration: Adding lightweight sentiment or interaction-pattern analysis to identify customer frustration and change response tone or escalation behavior based on it.
- Increased enterprise integration: Combining with CRM and ticketing systems to allow a complete and fluent handoff to human agents with conversation context.
- Scalability and optimization of deployment: Containerization and resource optimization to increase the number of calls that the system can handle and improve its performance.

These improvements are based on the current architecture and continue the focus on control, explainability, and cost-effectiveness of the system.

12. Conclusion

This project showcases the making of a smart, conversational voice agent to enhance customer care for small and mid-sized businesses in India. It is a movement away from rigid menu-based IVR to natural language voice interactions that shows how selective automation can cut costs while lifting the customer experience.

It is modular in setup, hybrid in intelligence mix, and structured in dialogue management, which contributes to predictability and transparency with easy evaluation. What follows is that the work sets a very good foundation for further advanced customer care and automation; it gives practical, real-world lessons in building effective conversational AI systems.

13. References

(1) FastAPI Documentation, Available: https://fastapi.tiangolo.com

(2) spaCy Documentation, Available: https://spacy.io

(3) Coqui TTS Documentation, Available: https://tts.readthedocs.io